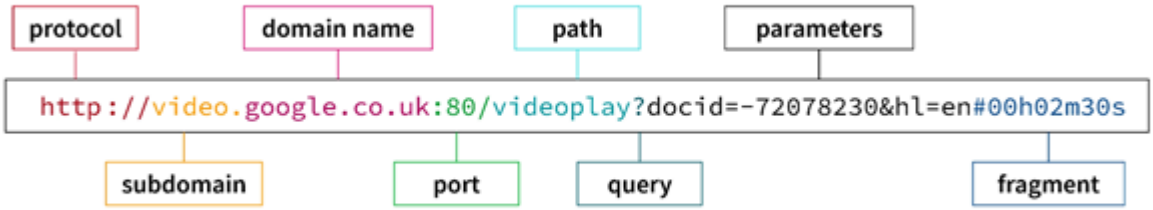
Node JS-2

URL module

The URL module contains functions that help in parsing a URL. In other words, we can split the different parts of a URL easily with the help of utilities provided by the URL module.



The syntax for including the url module in your application:

```
var url=require("url");
```

Parse an address with the **url.parse() method**, and it will return a URL object with each part of the address as properties.

url.parse() – This method takes the url string as a parameter and parses it. The url module returns an object with each part of the url as property of the object.

Syntax:

url.parse(url_string, parse_query_string, slashes_host)

Description of the parameters :

- **url_string :** <string> It is the URL string.
- **parse_query_string :** <boolean> It is a boolean value. **By default, its value is false**. If it is set to true, then the query string is also parsed into an object. Otherwise, the query string is returned as an unparsed string.
- **slashes_host :** <boolean> It is a boolean value. By default, its value is false. If it is set to true, then the token in between // and first / is considered host.

The returned object contains the following properties

- **protocol**: This specifies the protocol scheme of the URL (ex: https:).

- **slashes**: A Boolean value that specifies whether the protocol is followed by two ASCII forward slashes (//).
- **auth**: This specifies the authentication segment of the URL (ex: username: password).
- **username**: This specifies the username from the authentication segment.
- **password**: This specifies the password from the authentication segment.
- **host**: This specifies the host name with the port number.
- **hostname**: This specifies the host name excluding the port number.
- **port**: Indicates the port number.
- **pathname**: Indicates the URL path.
- **query**: This contains a parsed query string unless parsing is set to false.
- **hash**: It specifies the fragment portion of the URL **including the "#".**
- **href**: It specifies the complete URL.
- **origin**: It specifies the origin of the URL.
- **etc...**

Impact of parse_query_string value,

Defualt = false -> return values of query in string

Set true -> returns values of query in object

For example

```
var u=require("url");
var data = "http://localhost:8080/info?age=20";
var q1=u.parse(data);
   console.log(q1.query);
```

Property	url.parse(data)	url.parse(data, true)
query	'age=20'	{age: '20'}
Type	String	Object
Access age	Not direct so undefined	q.query.age → 20

Example

```
var u=require("url");
var addr="http://localhost:8080/default.htm?Name=Prof.Khushbu&Initial=PKP#MAM";
var q1=u.parse(addr,true);
   console.log(q1);
   console.log(q1.host);
   console.log(q1.pathname);
   console.log(q1.query);
```

Output:

```
Url {
  protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'localhost:8080',
  port: '8080',
  hostname: 'localhost',
  hash: '#MAM',
  search: '?Name=Prof.Khushbu&Initial=PKP',
  query: [Object: null prototype] { Name: 'Prof.Khushbu', Initial: 'PKP' },
  pathname: '/default.htm',
  path: '/default.htm?Name=Prof.Khushbu&Initial=PKP',
  href: 'http://localhost:8080/default.htm?Name=Prof.Khushbu&Initial=PKP#MAM'
}
localhost:8080
/default.htm
[Object: null prototype] { Name: 'Prof.Khushbu', Initial: 'PKP' }
#MAM
```

Task: Find a leap year from static url

```
var u=require("url");
var addr="http://localhost:8080/default.html?year=2025&month=feb";
var q=u.parse(addr,true);
console.log(q);
var qdata=q.query;
console.log(qdata.year);
if((qdata.year%4==0 && qdata.year % 100 !== 0) || (qdata.year % 400 === 0))
{
  console.log("Its a leap year")
}
else{
  console.log("Its not a leap year")
}
```

==========

Output:

```
Url {
  protocol: 'http:',
  slashes: true,
  auth: null,
  host: 'localhost:8080',
  port: '8080',
  hostname: 'localhost',
  hash: null,
  search: '?year=2025&month=feb',
  query: [Object: null prototype] { year: '2025', month: 'feb' },
  pathname: '/default.html',
  path: '/default.html?year=2025&month=feb',
  href: 'http://localhost:8080/default.html?year=2025&month=feb'
}
2025
Its not a leap year
```

- **Write a nodejs script to print query string of url on console as well as on file using ES6 callback.**

Case -1 Fetch query as an object. Need to convert query to string using stringify and then we can write it in file.

```
var u=require("url");
var fs=require("fs");
var adr1=" http://localhost:8080/default.html?year=2025&month=feb";
        var q1=u.parse(adr1,true);
        var qdata = JSON.stringify(q1.query)
        fs.writeFile("fsd2.txt",qdata,(err)=>{
                console.log("completed");});
```

Case-2 Fetch query as a string. We can directly write query string in a file. (Alternate approach)

```
var u=require("url");
var fs=require("fs");
var adr1=" http://localhost:8080/default.html?year=2025&month=feb";
        var q1=u.parse(adr1,false);        // false directly gives string instead of object
        var qdata=q1.query;
        fs.writeFile("fsd2.txt",qdata,(err)=>{
           console.log("completed");});
```

While running above code of url using .parse() it will give below warning.
DeprecationWarning: `url.parse()` behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API (**Web HyperText Application Technology Working Group- creates modern web standards)**
instead. CVEs are not issued for `url.parse()` vulnerabilities.
(Use `node --trace-deprecation ...` to show where the warning was created)

Deprecation means a feature is **old and not recommended to use**.
It still works, but may be **removed in future versions**

Option	Command / Code	Purpose
Disable Warning (Terminal)	node --no-deprecation filename.js	Hides deprecation warnings while running program
Suppress in Code	process.noDeprecation = true;	Disables warnings inside the program
Show Full Trace	node --trace-deprecation filename.js	Shows where the warning is generated

Refernce alternate approach Next Gen

```
const myURL =
new URL("http://localhost:8080/info.html?name='ABC'&Age=20#view")
console.log(myURL)
// observe object's individual properties

//To access query values
console.log("Name is:", myURL.searchParams.get("name"));
console.log("Age is:", myURL.searchParams.get("Age"));
```

Example

Write a Node.js program to parse a URL and calculate the average of three numbers passed as query parameters.

url: "http://localhost:8080/test?m1=50&m2=60&m3=70";

```
var url = require("url");
process.noDeprecation = true; // Only add to ignore warning in terminal
var adr = "http://localhost:8080/test?m1=50&m2=60&m3=70";

var q = url.parse(adr, true);
var m1 = parseInt(q.query.m1);
var m2 = parseInt(q.query.m2);
var m3 = parseInt(q.query.m3);
var avg = (m1 + m2 + m3) / 3;
console.log("Average:", avg);
```

HTTP Module: Render Response, Read HTML File Server, Routing

Node.js has a built-in module called HTTP, which allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP).

To include the HTTP module, use the require() method:

var http = require('http');

The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client.

node js provides capabilities to create your own web-server. Which will handle http requests asynchronously.

http.createServer()

This method turns your computer into an HTTP server and creates an HTTP Server object. That includes request and response parameters.

Syntax : http.createServer(**requestListener**)

requestListener*() function*.

- The requestListener is a function that is called each time the server gets a request.
- The requestListener function is passed as a parameter to the http.createServer() method.
- The requestListener function handles *requests* from the user, and also the *response* back to the user

The **function(req,res){…} passed** into the http.createServer() method, will be executed when someone tries to access the computer on specified port.

- "req" parameter is the request object representing the HTTP request from the client.
- "res" parameter is the response object representing the HTTP response that will be sent back to the client.

server.listen(port number, callback): method creates a listener on the specified port or path. You can specify any unused port no.

- port number: Specifies the port we want to listen to
- callbcak (optional): Specifies a function to be executed when the listener has been added.

Example to create server and print "Hello world" message in **index,js** file

```
var http = require('http');
var server = http.createServer(                    //create a server object
function (req, res) {
  res.write('Hello World!');          //write a response to the client
  res.end();                          //end the response can be empty or include string
}).listen(8080);                      //the server object listens on port 8080
//(or server.listen(5051) instead of listen());
```

Run file by **node index.js** in terminal and hit http://localhost:8080 on browser

Output on browser → http://localhost:8080/

Hello World!

Add an HTTP Header(To handle HTTP Requestes)

If the response from the HTTP server is supposed to be displayed as HTML, you should include an HTTP header with the correct content type:

Syntax: res.writeHead(status code, Type of header)

The first argument of the res.writeHead() method is the status code, 200 means that all is OK, the second argument is an object containing the response headers.

Status Code:

HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

1. **Informational responses (100 – 199)** For ex. 100 – continue, 101- Switching protocols etc.
2. **Successful responses (200 – 299)** For ex. 200 – Ok, 201 – Created, 202 – Accepted etc.
3. **Redirection messages (300 – 399)** For ex. 301 - Move permanently, 304 -Not modified etc.
4. **Client error responses (400 – 499)** For ex. 400 – Bad request, 402 – payment require, 403 – forbidden, 404 – page not found etc.
5. **Server error responses (500 – 599)** For ex 500 – Internal Server Error, 502–Bad Gateway , 503 – Service unavailable, 505 - HTTP Version Not Supported etc

Type of Header syntax : {"Content-Type": "MIME type"}

Name	MIME type
Plain text	text/plain
HyperText Markup Language (HTML)	text/html
JavaScript Object Notation (JSON)	application/json
JPEG Image	image/jpeg
Portable Network Graphics (PNG)	image/png
CSS	text/css
javascript	application/javascript

Example: Create server and print "Hello world" message in **h1 Tag**

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('<h1>Hello World!</h1>');
  res.end();
}).listen(8180);
```

Import module

```
var http = require('http');
```

You're using Node's built-in **HTTP module**
This lets you create a web server

Create server

```
http.createServer(function (req, res) {
```

This runs **every time a request comes in**

- req → request from browser
- res → response you send back

writeHead()

```
res.writeHead(200, { 'Content-Type': 'text/html' });
```

What this does:

```
1. Status Code : 200
```

Means: **OK / Success**

```
2. Headers :{ 'Content-Type': 'text/html' }
```

Tells browser:
"I am sending HTML, render it properly"
Without this:

- Browser may treat it as plain text
- HTML tags may not render

Write body

```
res.write('<h1>Hello World!</h1>');
```

This is the **actual data sent to browser**

- You can call .write() multiple times
- It sends data in chunks

End response

```
res.end();
```

This tells Node:
"I'm done — send everything now"

Flow:
Client (Browser) → Request → Server
Server:
  writeHead() → set status + headers
  write() → send data
  end() → finish response
Response goes back to browser

Listening to Port

```
}).listen(8180);
```

The server listens on port 8180.
You can access it in the browser using: **http://localhost:8180**

Content-Type header is mandatory for correct rendering. Without it, the browser guesses the format, which can lead to inconsistent results.

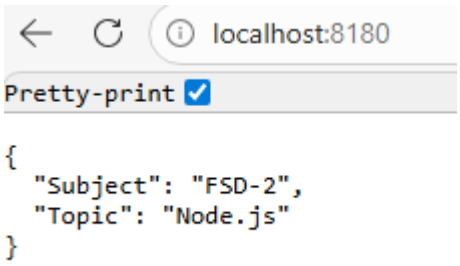
Browser does NOT read your mind. It depends on headers to understand data.

- Without header → unlabeled box
  → "What is this? Text? HTML? Image?"
- With header → labeled box
  → "This is HTML → render it"

Note: In above Example if you change MIME type in header from text/HTML to text/plain then utput will raw text instead of HTML.

JSON Example

```
var http = require('http');
http.createServer(function (req, res) {
  // Tell browser this is JSON
  res.writeHead(200, { 'Content-Type': 'application/json' });
  const data = {
    Subject: "FSD-2",
    Topic: "Node.js"
  };
  res.end(JSON.stringify(data));
}).listen(8180);
```



Request Url Approch

Query string

The function passed into the http.createServer() has a req argument that represents the request from the client, as an object (http.IncomingMessage object).
This object has a property called "url" which holds the part of the url that comes after the domain name

For example, if the user visits:

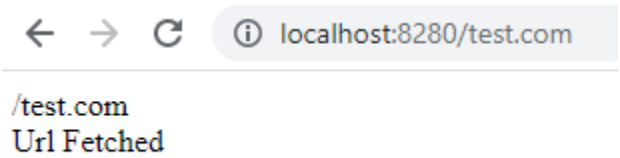
```
http://localhost:8280/about
```

Then:

```
req.url = "/about"
```

Example:

```
var http = require('http');
http.createServer(function (req, res) {
res.writeHead(200, {'Content-Type': 'text/html'});
res.write(req.url);
res.end("Url Fetched");
}).listen(8280);
```



/test.com
Url Fetched

Note:

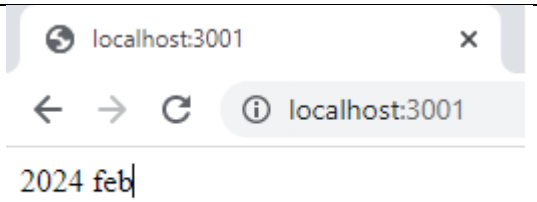
- If you change the text in res.write(), then you must run the code again to see the new text on the browser.
- If you don't specify res.end() then the browser will not stop progress and shows continuously loading.
- res.writeHead() Method require to specify content type on your server. Else it will automatically set content-type.
  For example: if First res.write() has plain text and second res.write() has HTML element. Then server will set plain content-type as text/plain and your html code will display as a plain text.

Get the details from query string

We can fetch the values from url query string as mentioned below using URL module.

1) Add static url in code and request server to display data of query string

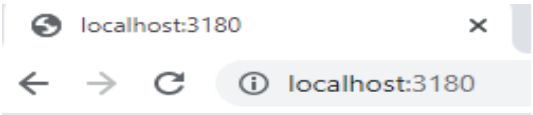
```
var http = require('http');
var url = require('url');
var addr="http://localhost:8080/default.html?year=2024&month=feb";
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
/*Use the url module to get the querystring*/
  var q = url.parse(addr, true).query;
/*Return the year and month from the query object: */
  var txt = q.year + " " + q.month;
  res.end(txt);
}).listen(3001);
```



2) Make changes in url at browser and request url to display data.

```
var http = require('http');
var url = require('url');

http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  var q = url.parse(req.url, true).query;
  var txt = q.year + " " + q.month;
  res.end(txt);
}).listen(3180);
```



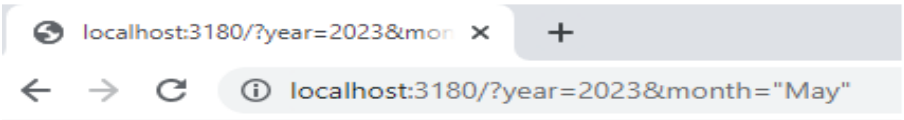


Image example

```
var http = require('http');
http.createServer(function (req, res) {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Hello</h1><img src="logo192.png" />');
  }
}).listen(8180);
```

Image is Broken in above example

- The img tag does not contain the image itself. It tells the browser to request the image separately from the server.
- When the page loads, the browser first requests the main page using /. The server sends the HTML response. Then the browser sees the image tag and makes another request for /logo192.png.
- If the server does not handle the /logo192.png request, it does not send any response. Because of this, the browser cannot load the image and shows it as broken.

Solution is below code in above example

```
var http = require('http');
var fs = require('fs');

http.createServer(function (req, res) {

  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Hello</h1><img src="logo192.png" />');
  }

  else if (req.url === '/logo192.png') {
    const img = fs.readFileSync('logo192.png');
    res.writeHead(200, { 'Content-Type': 'image/png' });
    res.end(img);
  }

}).listen(8180);
```

- The server must handle the image request and send the image file with the correct Content-Type (image/png).

Each resource like HTML, image, CSS, or JavaScript is a separate HTTP request and must be handled by the server.

Example:1
Create HTTP webpage on which home page display “Home page”, student page shows “Student page” and any other page shows “Page Not found”.
(Render Response & Routing)

```
var h=require("http");
var server=h.createServer(
   function(req,res)
   {
if(req.url=="/")
{
   res.writeHead(200,{"content-type":"text/html"});
   res.write("<b> Home page </b>");
   res.end();
}
else if(req.url=="/student")
{
   res.writeHead(200,{"content-type":"text/plain"}); //plain shows code as it is
   res.write("<i> Student page </i>");
   res.end();
}
else {
   res.writeHead(404,{"content-type":"text/html"});
   res.write("<h1> Page Not found </h1>");
   res.end("Thanks");
   }    });
server.listen(5001);
console.log("Thanks for run");
```

Example:2
Write node js script to perform tasks as asked.

1) **Create one page with two links (Home(/) and about(/about)).**
2) **Both pages must contain HTML type content and add required content on both the pages.**
3) **If user add any other URL path, then he/she will be redirected to page and plain message will be displayed of “Page not found”.**

```
var h=require("http");
var server=h.createServer(
   function(req,res)
   {
      if(req.url=="/")
      {
         res.writeHead(200,{"content-type":"text/html"});
```

```
            res.write("<h1> Home page </h1><div><ul><li><a href='/'>Home</a></li><li><a
href='/about'>About</a></li></ul>");
            res.end();
        }
        else if(req.url=="/about")
        {
            res.writeHead(200,{"content-type":"text/html"});
            res.write("<h1> About Page </h1>");
            res.end();
        }
        else
        {
            res.writeHead(404,{"content-type":"text/plain"});
            res.write("Page not found");
            res.end("\nPlease check the url");
        /* res.write("Bye");*/ //display nothing if you add any content after res.end

        }
    });
server.listen(5051);
console.log("Thanks!");
```

Example 3:
Create http webpage and pass JSON object on webpage. // JSON Response

```
var http=require("http");
var server=http.createServer(
function(req,res){
if(req.url=="/"){
const a={"Name":"ABC", "Age":35};
res.writeHead(200,{"content-type":"application/json"});
res.write("Thank you..!");
res.write(JSON.stringify(a));
res.end();
}});
server.listen(6008);
```

Example 4:

Create HTTP webpage on which home page will fetch json data, about page shows centrally aligned message "Hello from class" in red colour and cyan background using internal css styling. And any other page shows "404 error". (Render Response & Routing)

```
const http=require("http")
http.createServer((req,res)=>{
if(req.url=="/"){const data={
"name":"ABC",
"age":30}
res.writeHead(200,{"content-Type":"application/json"})
res.write(JSON.stringify(data))
res.end()}
else if(req.url=="/about"){
res.writeHead(200,{"content-Type":"text/html"})
res.write(`<!DOCTYPE html>
<html lang="en"><head><style>body {
background-color: cyan;
text-align: center;}
h1 {color: red;}
</style> </head>
<body>
<h1>Hello from class</h1>
</body>
</html>`)
res.end()
}
else{
res.writeHead(404,{"content-Type":"text/html"});
res.end("<h1>404 error</h1>")
}
}).listen(3000)
console.log("server running")
```

Example 5:

Write a nodejs program <u>which fetch filename from requested url</u> and print that file's data on http web server.

```
var h=require("http");
var ps=require("fs");
var u=require("url");
var server=h.createServer(function(req,res) {
        var q=u.parse(req.url,true);

   data=ps.readFileSync("."+q.pathname);
    res.writeHead(200,{"content-type":"text/html"}); //text/plain gives program
```

```
//res.writeHead(200,{"content-type":"text/plain"}); gives content of file(Whole program will display in port)

res.write(data);
res.end();
});
server.listen(6052);
```

Note: # data=ps.readFileSync("."+q.pathname); - The dot (.) refers to the current directory. We can also use without "." By writing entire path.
Form.html must be in same folder where this file stored to observe feature of "."

Type in URL after running node script: http://localhost:6052/form.html

Example 6:
load a simple html file from variable on nodejs web server and print its content as html content.

```
var h=require("http");
var ps=require("fs");
var u=require("url");
var addr="http://localhost:8080/16.html"; (html file name)
var q=u.parse(addr,true);
data=ps.readFileSync("."+q.pathname);
h.createServer( function(req,res)    {
   res.writeHead(200,{"content-type":"text/html"});
   res.end(data);
}).listen(6051);
```

Example 7:

Write a nodejs program load a simple html file defined as static on nodejs web server and print its content as html content.

```
var http = require("http");
var fs = require("fs");
var url = require("url");
var addr = "http://localhost:6051/7.html";
```

```
var q = url.parse(addr, true);
http.createServer(function (req, res) {
    fs.readFile("." + q.pathname, function (err, data) {
        if (err) {
            res.writeHead(404, { "Content-Type": "text/plain" });
            res.end("File Not Found");
        } else {
            res.writeHead(200, { "Content-Type": "text/html" });
            res.end(data);
        }
    });
}).listen(6051);
```

Example 8:

Write a nodejs program load a simple html file on nodejs web server and print its content as html content.

```
var http = require("http");
var fs = require("fs");
http.createServer(function (req, res) {
    var file = "." + (req.url === "/" ? "/7.html" : req.url);
    fs.readFile(file, function (err, data) {
        if (err) {
            res.writeHead(404, { "Content-Type": "text/plain" });
            res.end("File Not Found");
        } else {
            res.writeHead(200, { "Content-Type": "text/html" });
            res.write(data);
            res.write("<h1>Hello</h1>");
            res.end();
        }
    });
}).listen(6055);
```

Explanation

- Uses req.url to load file dynamically
- / loads default file (7.html)
- Other URLs load corresponding files
- Adds extra HTML (Hello) in response

Example 9:

Write node js script to fetch values from url given below and display output as asked.
"https://www.google.com/exam.txt?c1=Hello&c2=FSD2 T1 Test&c3=Welcome to LJU#AllTheBest"
1) Data must be written as below in file named "exam.txt". File name must be fetched from the url given above.
Output:
Hello!
Welcome to LJU FSD2 T1 Test
#AllTheBest
2) Read content from file "exam.txt" and send response to server and display data in "/" page in same format as above but in H1 tag and in red color.
3) If any other page is requested it shows "Page not found" message in plain text.

```
const http = require("http");
const fs = require("fs");
const url = require("url");

const url1 =
"https://www.google.com/exam1.txt?c1=Hello&c2=FSD2+T1+Test&c3=Welcome+to+LJU#A
llTheBest";

const parsedUrl = url.parse(url1,true);
console.log(parsedUrl.pathname)

const c1 = parsedUrl.query.c1;
const c2 = parsedUrl.query.c2;
const c3 =  parsedUrl.query.c3;
const hash =  parsedUrl.hash;
const filename = "."+parsedUrl.pathname

const fileContent = c1+"!\n"+c3+" "+c2+"\n"+hash;

fs.writeFileSync(filename, fileContent);

const server = http.createServer((req, res) => {
   if (req.url === "/") {
      data= fs.readFileSync(filename, "utf-8")
      res.writeHead(500, { "Content-Type": "text/html" });
      res.end("<h1 style='color:red'><pre>"+data+"<pre></h1>");
   }
   else {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("Page not found");
   }
});

server.listen(3000)
```

Example 10:

Create HTTP webpage on which Home page display "Welcome to Log in page" in blue color and font size must be 32px, Login page shows one HTML file from static URL having Form with detail for Username, Password, submit and reset button, Gallery page reflect one Image "hello.jpg" and any other page shows "Page Not found". Write all necessary files to perform task. (Image already exist in same folder)

```
var http = require("http");
var fs = require("fs");
var server = http.createServer(function (req, res) {
   // Home Page
   if (req.url == "/") {
      res.writeHead(200, { "Content-Type": "text/html" });
      res.write("<h1 style='color:blue;font-size:32px;'>Welcome to Log in page</h1>");
      res.end();
   }
   // Login Page
   else if (req.url == "/login") {
      fs.readFile("logi.html", function (err, data) {
         if (err) {
            res.writeHead(500);
            return res.end("Error loading file");
         }
         res.writeHead(200, { "Content-Type": "text/html" });
         res.write(data);
         res.end();
      });
   }
   // Gallery Page
   else if (req.url == "/gallery") {
      fs.readFile("logo192.png", function (err, data) {
         if (err) {
            res.writeHead(500);
            return res.end("Error loading image");
         }
         res.writeHead(200, { "Content-Type": "image/jpeg" });
         res.end(data);
      });
   }
   // Page Not Found
   else {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("Page Not Found");
   }
});
server.listen(5051, () => {
   console.log("Server running at http://localhost:5051");
});
```